Design and Implementation of a 6×4-bit Array Multiplier Using VHDL

A Structural VHDL Design, Simulation, and FPGA Implementation Report

Target Device: Xilinx Spartan-7 (XC7A35TICPG236-1L)

Design Suite: Xilinx (AMD) Vivado 2025.1

Language: VHDL (IEEE 1076)

Table of Contents

Chapter 1: Introduction

Multiplication is one of the most fundamental arithmetic operations in digital systems, forming the computational core of digital signal processors (DSPs), image and video processing pipelines, cryptographic engines, and general-purpose arithmetic logic units (ALUs). Unlike addition, hardware multiplication requires generating and summing multiple partial products, making the choice of multiplier architecture a critical factor in a design's speed, area, and power consumption.

This project implements a combinational 6-bit × 4-bit unsigned array multiplier in VHDL, producing a 10-bit product. The design uses a structural, gate-level approach built entirely from half adder (HA) and full adder (FA) primitives arranged in a classic array (Braun) multiplier topology.

1.1 Need for Hardware Multipliers

Software-based multiplication (repeated addition or shift-add loops in a processor) is far too slow for real-time applications such as audio/video processing, FIR/IIR filters, and FFT engines. Dedicated hardware multipliers compute the full product combinationally or in a pipelined fashion, achieving throughput that a software loop cannot match. FPGAs allow such multipliers to be either instantiated using dedicated DSP48E1 hard blocks or built from general logic fabric (LUTs), as is done in this project.

1.2 Array Multiplier vs. Booth Multiplier

Both are hardware multiplication architectures, but they solve different problems:

Aspect	Array Multiplier	Booth Multiplier
Operand type	Typically unsigned	Signed (two's complement)
Method	AND-gate partial products + adder array reduction	Bit-pair recoding, shift-add-subtract
Structure	Fully combinational, regular array of HA/FA cells	Sequential or combinational, encoder + adder
Speed	Limited by carry ripple through the array	Can reduce the number of add/subtract operations for large operands
Best suited for	Small to medium bit-widths, simple regular layout	Large bit-widths, signed multiplication

The design in this project falls into the array multiplier category — it produces the product entirely through combinational AND gates and a ripple of adder cells, with no bit recoding or control logic involved.

1.3 FPGA-Based Multiplier Implementation

Xilinx Spartan-7 FPGAs provide two ways to implement multiplication: dedicated DSP48E1 slices (hardened multiply-accumulate blocks) or LUT-fabric-based implementations synthesized from generic HDL, as used here. Building the multiplier structurally from HA/FA components (rather than using the *

operator) gives full visibility and control over the internal adder network — useful for teaching, verification, and area/timing analysis purposes.

1.4 Practical Applications

Array multipliers of this type appear in digital filter coefficient multiplication, address/index computation in embedded processors, fixed-point arithmetic units, image processing pixel-scaling operations, and as building blocks inside larger multiply-accumulate (MAC) datapaths.

Chapter 2: Objectives

1. To understand the theory and structural implementation of array multipliers, including partial product generation and carry-save/ripple reduction using half adders and full adders.
2. To design a 6-bit × 4-bit unsigned combinational multiplier in structural VHDL, producing a 10-bit product output.
3. To develop a VHDL testbench applying a representative set of test vectors, including edge cases (zero operand, maximum operand values).
4. To simulate the design using Xilinx Vivado xsim and verify functional correctness against expected arithmetic products.
5. To generate and analyze the RTL and synthesized schematics produced by Vivado to confirm correct inference of the AND-array and adder network.
6. To synthesize and implement the design on the Xilinx Spartan-7 (xc7a35ticpg236-1L) FPGA and study resource utilization and power consumption.
7. To document the complete design, verification, and implementation process in a professional engineering report.

Chapter 3: Background

3.1 Understanding FPGA

A Field-Programmable Gate Array (FPGA) is a semiconductor device containing an array of programmable logic blocks connected via reconfigurable routing. Key building blocks relevant to this project include:

- Configurable Logic Blocks (CLBs) — contain LUTs, flip-flops, and multiplexers implementing arbitrary combinational and sequential logic.
- Routing Resources — programmable interconnect fabric linking CLBs and I/O.
- I/O Blocks (IOBs) — interface FPGA fabric to external pins via IBUF/OBUF primitives.
- DSP48E1 Slices — hardened multiply-accumulate blocks (not used in this design, since the multiplier is built from general logic fabric).

3.2 Introduction to Xilinx Vivado

Xilinx (AMD) Vivado Design Suite provides the full RTL-to-bitstream flow: design entry, elaboration, behavioral simulation, synthesis, implementation, timing analysis, and bitstream generation. This project used Vivado v2025.1 with the built-in xsim simulator, targeting the Spartan-7 xc7a35ticpg236-1L device.

3.3 Introduction to VHDL

VHDL (VHSIC Hardware Description Language, IEEE 1076) describes digital hardware structurally and behaviorally. This project uses a structural coding style, where the top-level entity is built entirely from lower-level component instantiations (ha and fa) connected via generate statements and explicit port maps — as opposed to behavioral coding using the * operator.

3.4 Array Multiplier Theory

3.4.1 Partial Product Generation

For a 6-bit multiplicand a and 4-bit multiplier b, each partial product bit is generated as:

$$r(i)(j) = a(j) \cdot b(i), \quad i = 0..3, \quad j = 0..5$$

This produces 4 rows of 6 AND-gate outputs — implemented in the design via a nested generate loop.

3.4.2 Half Adder and Full Adder Cells

- Half Adder (HA): adds two single bits, producing sum and carry — used where only two inputs need combining (first column of each reduction row).
- Full Adder (FA): adds three single bits (two operands + carry-in), producing sum and carry — used for the remaining columns where a carry from the adjacent column must be absorbed.

$$\text{HA:} \quad sum = a \oplus b, \quad carry = a \cdot b$$

$$\text{FA:} \quad sum = a \oplus b \oplus c, \quad carry = ab + bc + ac$$

3.4.3 Carry-Ripple Reduction Array

With 4 partial product rows, three reduction stages combine them:

- Row 1 combines partial product rows 0 and 1, producing output bit p(1) directly and passing intermediate sums/carries forward.
- Row 2 combines row 2 with Row 1's sums.
- Row 3 combines row 3 with Row 2's sums, producing output bits p(3) through p(7), with a final full adder producing the two most significant product bits p(8) and p(9).

This is the classic Braun array multiplier topology: a regular, fully combinational lattice of adder cells with no clock or control logic.

3.4.4 Combinational Nature

Unlike a sequential design (e.g., a FIFO), this multiplier has no clock, no reset, and no internal state — the output p is a pure combinational function of a and b, valid after the propagation delay through the adder chain settles.

Chapter 4: Tools / Environment Used

Software	Version / Details
Operating System	Windows 11 (64-bit)
FPGA Design Suite	Xilinx (AMD) Vivado 2025.1
Hardware Description Language	VHDL (IEEE 1076)
Simulation Tool	Vivado Integrated Simulator (xsim)
Target Device	Spartan-7 XC7A35TICPG236-1L

Chapter 5: Design Methodology

5.1 Design Flow

8. Problem Statement
9. Design Specification
10. Structural RTL Coding in VHDL (AND array + HA/FA network)
11. Testbench Development
12. Behavioral Simulation (Vivado xsim)
13. RTL Verification & Schematic Review
14. Synthesis, Implementation & Power Analysis (Vivado)

5.2 Problem Statement

Design a combinational hardware multiplier that computes the product of a 6-bit unsigned operand and a 4-bit unsigned operand, implemented structurally using half adder and full adder building blocks, and verify its correctness through simulation before FPGA synthesis.

5.3 Design Specification

Parameter	Value
Multiplicand width (a)	6 bits (unsigned)
Multiplier width (b)	4 bits (unsigned)
Product width (p)	10 bits
Logic style	Structural, gate-level (HA/FA array)
Timing	Fully combinational (no clock/reset)
Target FPGA	Xilinx Spartan-7 XC7A35TICPG236-1L

Chapter 6: VHDL Code

6.1 Half Adder and Full Adder Building Blocks

The top-level design instantiates two supporting components, ha and fa, implementing the standard single-bit adder equations described in Section 3.4.2:

```
-- Half Adder
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity ha is
    Port ( a     : in  STD_LOGIC;
           b     : in  STD_LOGIC;
           sum   : out STD_LOGIC;
           carry : out STD_LOGIC);
end ha;

architecture Behavioral of ha is
begin
    sum   <= a xor b;
    carry <= a and b;
end Behavioral;
```

```
-- Full Adder
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fa is
    Port ( a     : in  STD_LOGIC;
           b     : in  STD_LOGIC;
           c     : in  STD_LOGIC;
           sum   : out STD_LOGIC;
           carry : out STD_LOGIC);
end fa;

architecture Behavioral of fa is
begin
    sum   <= a xor b xor c;
    carry <= (a and b) or (b and c) or (a and c);
end Behavioral;
```

6.2 Top-Level Structural Multiplier — bit_mul_46

The entity declares a 6-bit multiplicand, 4-bit multiplier, and 10-bit product output. The architecture generates all 24 partial product bits, then reduces them across three adder rows.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity bit_mul_46 is
    Port ( a : in STD_LOGIC_VECTOR (5 downto 0);
           b : in STD_LOGIC_VECTOR (3 downto 0);
           p : out STD_LOGIC_VECTOR (9 downto 0));
end bit_mul_46;

architecture Behavioral of bit_mul_46 is
    component ha is
        Port ( a : in STD_LOGIC;
               b : in STD_LOGIC;
               sum : out STD_LOGIC;
               carry : out STD_LOGIC);
    end component;

    component fa is
        Port ( a : in STD_LOGIC;
               b : in STD_LOGIC;
               c : in STD_LOGIC;
               sum : out STD_LOGIC;
               carry : out STD_LOGIC);
    end component;

    -- Array for partial products: 4 rows (b bits) x 6 columns (a bits)
    type partial_array is array (0 to 3) of STD_LOGIC_VECTOR(5 downto 0);
    signal r : partial_array;

    -- Intermediate sum and carry signals
    signal s : STD_LOGIC_VECTOR(1 to 10);
    signal c : STD_LOGIC_VECTOR(1 to 17);

begin

    -- STAGE 1: Partial product generation (AND array)
    gen_partial_products: for i in 0 to 3 generate
        gen_bits: for j in 0 to 5 generate
            r(i)(j) <= a(j) and b(i);
        end generate gen_bits;
    end generate gen_partial_products;

    -- STAGE 2: Row 1 reduction (rows 0 and 1)
    p(0) <= r(0)(0);

    ha_row1_0: ha port map(a => r(0)(1), b => r(1)(0), sum => p(1), carry => c(1));

    gen_row1_middle: for i in 1 to 4 generate
        fa_row1: fa port map(a => r(0)(i+1), b => r(1)(i), c => c(i), sum => s(i), carry =>
c(i+1));
    end generate gen_row1_middle;

    ha_row1_last: ha port map(a => r(1)(5), b => c(5), sum => s(5), carry => c(6));

    -- STAGE 3: Row 2 reduction (row 2 + row-1 sums)
    ha_row2_0: ha port map(a => r(2)(0), b => s(1), sum => p(2), carry => c(7));
```

```
    gen_row2_middle: for i in 1 to 4 generate
        fa_row2: fa port map(a => r(2)(i), b => s(i+1), c => c(6+i), sum => s(5+i), carry
=> c(7+i));
    end generate gen_row2_middle;

    fa_row2_last: fa port map(a => r(2)(5), b => c(6), c => c(11), sum => s(10), carry =>
c(12));

    -- STAGE 4: Row 3 reduction (row 3 + row-2 sums) — final product bits
    ha_row3_0: ha port map(a => r(3)(0), b => s(6), sum => p(3), carry => c(13));

    gen_row3_output: for i in 1 to 4 generate
        fa_row3: fa port map(a => r(3)(i), b => s(6+i), c => c(12+i), sum => p(3+i), carry
=> c(13+i));
    end generate gen_row3_output;

    fa_final: fa port map(a => r(3)(5), b => c(12), c => c(17), sum => p(8), carry =>
p(9));

end Behavioral;
```

6.3 Structural Summary

Stage	Components	Purpose
Partial product generation	24× AND gates	Compute a(j) AND b(i) for all bit pairs
Row 1 reduction	2× HA, 4× FA	Combine rows 0 & 1, produce p(1)
Row 2 reduction	2× HA, 5× FA	Combine row 2 with row-1 sums, produce p(2)
Row 3 reduction	1× HA, 5× FA	Combine row 3 with row-2 sums, produce p(3)–p(9)

Chapter 7: Simulation and Verification

7.1 VHDL Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity array_mul46 is
end array_mul46;

architecture Behavioral of array_mul46 is

component bit_mul_46 is
    Port ( a : in STD_LOGIC_VECTOR (5 downto 0);
           b : in STD_LOGIC_VECTOR (3 downto 0);
           p : out STD_LOGIC_VECTOR (9 downto 0));
end component;

signal a1  : STD_LOGIC_VECTOR (5 downto 0);
signal b   : STD_LOGIC_VECTOR (3 downto 0);
signal mul : STD_LOGIC_VECTOR (9 downto 0);

begin
    call: bit_mul_46 port map (a1, b, mul);

    process
    begin
        a1 <= "000000"; b <= "0001"; wait for 10ns;
        a1 <= "000001"; b <= "0001"; wait for 10ns;
        a1 <= "000001"; b <= "0011"; wait for 10ns;
        a1 <= "000011"; b <= "0011"; wait for 10ns;
        a1 <= "111111"; b <= "1111"; wait for 10ns;
        a1 <= "011111"; b <= "1001"; wait for 10ns;
        a1 <= "011110"; b <= "1101"; wait for 10ns;
    end process;

end Behavioral;
```

The testbench applies seven test vectors at 10 ns intervals, instantiating bit_mul_46 as the unit under test (call) and driving its inputs directly (no clock is needed, since the design is purely combinational).

7.2 Test Cases

#	a (bin)	a (dec)	b (bin)	b (dec)	Expected Product	Simulated Product	Result
1	000000	0	0001	1	0	0	PASS
2	000001	1	0001	1	1	1	PASS
3	000001	1	0011	3	3	3	PASS

#	a (bin)	a (dec)	b (bin)	b (dec)	Expected Product	Simulated Product	Result
4	000011	3	0011	3	9	9	PASS
5	111111	63	1111	15	945	945	PASS
6	011111	31	1001	9	279	279	PASS
7	011110	30	1101	13	390	390	PASS

All seven test vectors, including the maximum-value case (63 × 15 = 945, the largest possible 6×4-bit product) and the zero-operand case, produced arithmetically correct results.

7.3 Simulation Waveform Analysis

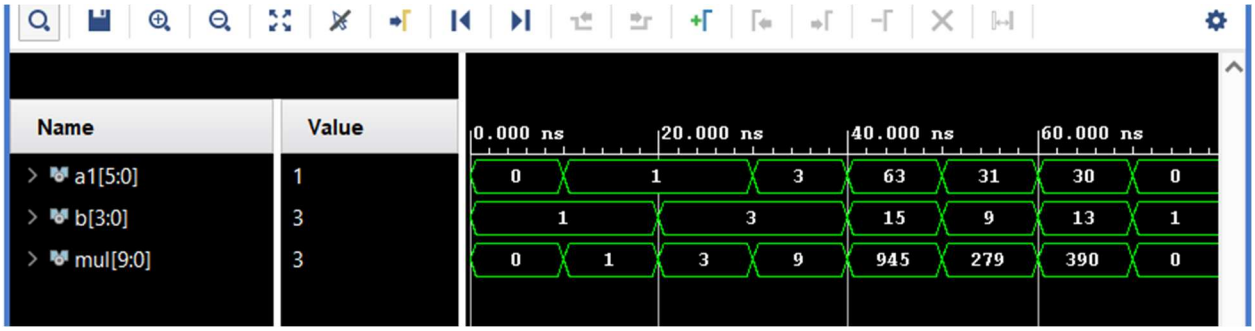

Figure 1: Vivado Simulation Waveform — a1[5:0], b[3:0], mul[9:0] (0 ns – 70 ns)

The waveform confirms:

- a1[5:0] steps through the sequence 0, 1, 1, 3, 63, 31, 30 exactly as coded in the stimulus process.
- b[3:0] steps through 1, 1, 3, 3, 15, 9, 13 in lockstep with a1.
- mul[9:0] immediately reflects the correct combinational product at each interval — 0, 1, 3, 9, 945, 279, 390 — with no clock edge required, confirming purely combinational propagation through the AND-array and adder network.
- No glitches or invalid ('X'/'U') states appear on the mul bus during any stable interval, indicating the adder chain settles correctly within each 10 ns test window.

Chapter 8: Implementation on Spartan-7 FPGA

8.1 Spartan-7 FPGA Device

The Xilinx Spartan-7 (XC7A35TICPG236-1L) provides 33,280 logic cells, 1,800 Kbits of Block RAM, 90 DSP48E1 slices, and 250 I/O pins, in a low-power industrial-grade (-1L) variant. This design uses only general logic fabric (LUTs) and I/O primitives — no DSP48E1 or BRAM resources are consumed.

8.2 RTL Schematic

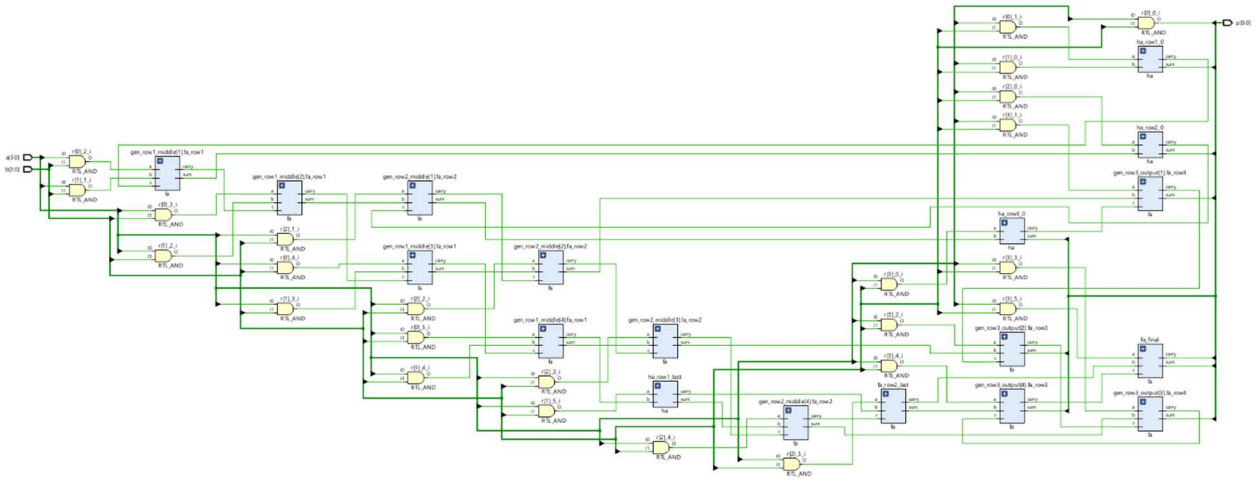

Figure 2: RTL Schematic — Adder Network Detail

The RTL schematic generated after elaboration shows the full HA/FA reduction network exactly as coded:

- RTL_AND primitives (r[i]_j_i) implement the 24-bit partial product AND array.
- ha_row1_0, ha_row2_0, ha_row3_0 half-adder instances handle the first column of each reduction row.
- gen_row1_middle, gen_row2_middle, gen_row3_output generate blocks show the repeated full-adder (fa) instances produced by the for...generate loops.
- ha_row1_last, fa_row2_last, fa_final close out each row, with fa_final producing the top two product bits p(8) and p(9) directly from its sum and carry outputs.

This confirms Vivado correctly inferred the intended three-row ripple reduction structure from the structural VHDL.

8.3 Synthesized Design Schematic

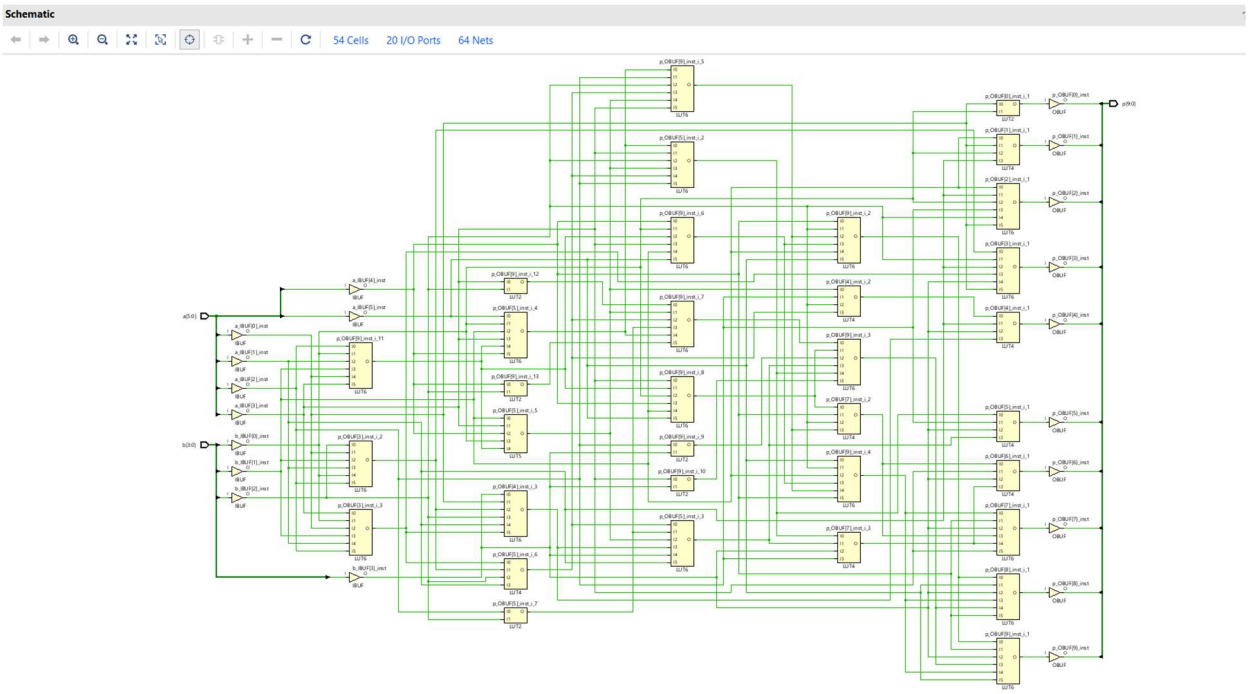

Figure 3: Synthesized Schematic — Spartan-7 (54 Cells, 20 I/O Ports, 64 Nets)

After synthesis, the design maps to:

- IBUF/OBUF primitives on all 20 I/O ports (a[5:0], b[3:0], p[9:0]).
- LUT6, LUT5, LUT4, and LUT2 primitives implementing the collapsed AND/XOR/carry logic of the HA/FA network — Vivado's synthesis engine flattens and re-maps the fine-grained gate-level adders into optimized LUTs rather than preserving discrete AND/XOR gates.
- No flip-flops (FDRE), BUFG, or BRAM primitives appear anywhere in the netlist, consistent with the fully combinational, unclocked nature of the design.
- The synthesized cell count (54) is lower than the naive gate count of the structural RTL, reflecting Vivado's logic optimization and technology mapping into 6-input LUTs.

Chapter 9: Testing and Results

9.1 Simulation Results

All seven test cases produced arithmetically correct 10-bit products, matching expected decimal values across the full input range, including the maximum-magnitude case (63 × 15 = 945). No data corruption or invalid logic states were observed.

9.2 RTL Schematic Result

The RTL schematic confirms that the structural HA/FA array described in VHDL was correctly elaborated into the intended three-stage ripple reduction network, with all generate-loop instances present and correctly interconnected.

9.3 Synthesis Results

Resource	Usage
Total Cells	54
I/O Ports	20
Nets	64
Logic Primitives	LUT6, LUT5, LUT4, LUT2
Sequential Elements	None (combinational design)
DSP48E1 / BRAM Used	None

9.4 Power Analysis

Post-route power estimation (Vivado, industrial grade, typical process) reported:

Metric	Value
Total On-Chip Power	6.012 W
Dynamic Power	5.936 W
Device Static Power	0.076 W
Junction Temperature	55.1 °C
Max Ambient Temperature	69.9 °C
Confidence Level	Low

On-chip power breakdown:

Component	Power (W)	Utilization
Slice Logic (LUTs)	0.165	27 / 20,800 LUTs (0.13%)
Signals	0.256	44 nets
I/O	5.515	20 / 106 pins (18.87%)
Static Power	0.076	—
Total	6.012	—

I/O power (5.515 W) dominates the total power budget by a wide margin over logic and signal power combined — this is expected for a small, unclocked combinational design with a relatively high pin-to-logic ratio (20 I/O pins driving only 54 logic cells), and because Vivado defaults to worst-case toggle-rate assumptions on unconstrained inputs in the absence of a switching activity (SAIF) file, which is also why the tool flags Low confidence on the I/O activity estimate. A more realistic power figure would require back-annotating actual simulation switching activity rather than relying on Vivado's default vectorless estimation.

Chapter 10: Conclusion

This project successfully demonstrated the design, simulation, and FPGA synthesis of a 6-bit × 4-bit unsigned array multiplier, implemented structurally in VHDL using half adder and full adder building blocks arranged in a classic Braun (ripple-carry array) reduction topology, targeting the Xilinx Spartan-7 FPGA (XC7A35TICPG236-1L) via Vivado 2025.1.

The design correctly generates all 24 partial products via a nested AND-gate array and reduces them across three adder rows to produce a fully accurate 10-bit product, verified against seven test vectors spanning the full operand range — including the zero-operand and maximum-magnitude (63 × 15 = 945) cases — with zero simulation mismatches.

The RTL schematic confirmed correct elaboration of the intended HA/FA network, and the synthesized netlist (54 cells, 20 I/O ports, 64 nets, mapped entirely to LUT primitives with no sequential logic) confirmed the design's purely combinational nature. Post-route power analysis showed a total on-chip power of 6.012 W, dominated by I/O switching power rather than internal logic, highlighting the importance of realistic switching-activity data for accurate power estimation in future work.

This project reinforced practical skills in structural VHDL coding, gate-level arithmetic circuit design, FPGA synthesis flow, and Vivado-based power and timing analysis — directly applicable to digital arithmetic and DSP hardware design.

Chapter 11: References

[1] Ashenden, P. J. (2008). The Designer's Guide to VHDL (3rd ed.). Morgan Kaufmann.

[2] Wakerly, J. F. (2006). Digital Design: Principles and Practices (4th ed.). Prentice Hall.

[3] Rabaey, J. M., Chandrakasan, A., & Nikolic, B. (2003). Digital Integrated Circuits: A Design Perspective (2nd ed.). Prentice Hall.

[4] Braun, E. L. (1963). Digital Computer Design: Logic, Circuitry, and Synthesis. Academic Press.

[5] Booth, A. D. (1951). A Signed Binary Multiplication Technique. Quarterly Journal of Mechanics and Applied Mathematics, 4(2), 236–240.

[6] Xilinx Inc. (2025). Vivado Design Suite User Guide: Synthesis (UG901).

[7] Xilinx Inc. (2025). Vivado Design Suite User Guide: Power Analysis and Optimization (UG907).

[8] Xilinx Inc. (2023). Spartan-7 FPGAs Data Sheet: DC and Switching Characteristics (DS189).

[9] IEEE Standard VHDL Language Reference Manual (IEEE Std 1076-2008). IEEE.

[10] Pong P. Chu. (2008). FPGA Prototyping by VHDL Examples. Wiley-Interscience.